TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM

KHOA KỸ THUẬT MÁY TÍNH

IT012 – TỔ CHỨC VÀ CẤU TRÚC MÁY TÍNH II

LAB 03

LÀM QUEN VỚI MARS: THANH GHI, TẬP LỆNH SỐ HỌC VÀ LỜI GỌI HỆ THỐNG

Họ và tên	Trần Yến Như	MSSV	25521347
Lớp	IT012.R11.2	Ngày thực hành	26/10/2026
Tự chấm	10	Link git	https://github.com/TranYenNhu25521347/IT012.R11.2_Lab

1. Tổng quan buổi thực hànhzz

Mục đích. Buổi thực hành này giúp sinh viên làm quen với cỗ máy nằm bên dưới mọi chương trình đã từng sử dụng. Xuất phát từ một câu lệnh số học thông thường như `a = b + c`, sinh viên sẽ thấy bộ xử lý lưu các số trong thanh ghi như thế nào, một lệnh hợp ngữ làm việc với ba thanh ghi ra sao, lệnh đó được lưu thành một số 32 bit duy nhất như thế nào, và một chương trình đang chạy giao tiếp với bàn phím và màn hình thông qua lời gọi hệ thống ra sao. Mỗi khái niệm mới đều được giải thích trước rồi mới thực hành.

> **Tình huống: phép tính bên trong công tơ điện thông minh**
>
> Công tơ điện của một hộ gia đình lưu tổng điện năng đã tiêu thụ, tính bằng kilowatt-giờ (kWh). Đầu mỗi tháng, công ty điện lực ghi lại chỉ số công tơ. Điện năng tiêu thụ trong một tháng là hiệu của hai chỉ số liên tiếp, còn điện năng tiêu thụ trong nhiều tháng là tổng của các hiệu theo tháng. Bộ xử lý nhỏ bên trong công tơ thông minh thực hiện đúng các phép trừ và phép cộng này trên các số nằm trong thanh ghi, sau đó báo cáo kết quả. Mọi chỉ số dùng trong bài thực hành này đều là số liệu giả định.

Sau buổi thực hành, sinh viên có thể:

- Mở, assemble, chạy, chạy từng bước và đặt lại một chương trình trong MARS; xác định được các khung Registers, Text Segment, Data Segment và Run I/O;
- Gọi tên các thanh ghi MIPS chính (`$zero`, `$t0`–`$t9`, `$s0`–`$s7`, `$v0`, `$a0`, `pc`) và đọc nội dung của chúng ở dạng thập phân và hexa;

- Chuyển các câu lệnh số học đơn giản như `a = b + c` thành lệnh MIPS dùng `add`, `sub` và `addi`, trong đó thanh ghi đóng vai trò biến;
- Mô tả cách một lệnh hợp ngữ được lưu thành từ mã máy 32 bit và đọc được số hiệu thanh ghi nằm trong đó;
- Dùng lời gọi hệ thống để in chuỗi, in số nguyên và đọc một số nguyên từ bàn phím;
- Dự đoán, quan sát và giải thích hành vi của một chương trình hợp ngữ ngắn.

Phần	Nội dung	Thời gian gợi ý	Điểm
Phần 0	Làm quen với MARS	20 phút	–
TN 1	Thí nghiệm 1 – Làm quen với thanh ghi	30 phút	2
TN 2	Thí nghiệm 2 – Từ `a = b + c` đến `$t0 = $t1 + $t2`	45 phút	3
TN 3	Thí nghiệm 3 – Nhập, xuất và lời gọi hệ thống	50 phút	3,5
Kết	Tổng kết và nộp bài	15 phút	1,5
–	Gỡ lỗi, kiểm tra và rà soát	20 phút	–
Tổng		180 phút	10

Quy ước làm bài

• Ô nền vàng là chỗ để ghi câu trả lời hoặc giá trị. Khung nét đứt là chỗ để dán ảnh chụp màn hình.

• Mã nguồn được đặt trong khung màu xám, font đơn cách. Trong mã nguồn, mọi nội dung sau dấu `#` là chú thích. Dòng bắt đầu bằng `# >>>` đánh dấu chỗ sinh viên tự viết mã.

• Mỗi câu hỏi được gắn nhãn theo mức nhận thức mà câu đó yêu cầu: *Hiểu*, *Vận dụng*, *Phân tích* hoặc *Đánh giá*.

• Nhiều hoạt động theo chu trình DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH. Trước hết ghi lại điều mình mong đợi (DỰ ĐOÁN), sau đó chạy chương trình và ghi lại điều thực sự xảy ra (QUAN SÁT), cuối cùng giải thích sự khác biệt (GIẢI THÍCH). Dự đoán sai không bị trừ điểm; điểm nằm ở phần giải thích.

• Đánh dấu vào ô ☐ trong danh sách kiểm tra khi đã hoàn thành mục tương ứng.

PHẦN 0 — Làm quen với MARS (20 phút)

MARS (MIPS Assembler and Runtime Simulator, phiên bản 4.5) là chương trình Java mô phỏng một bộ xử lý MIPS. MARS cho phép chạy chương trình hợp ngữ và theo dõi thanh ghi, bộ nhớ thay đổi qua từng lệnh mà không cần phần cứng MIPS thật.

Bảng 2. Danh sách kỹ năng

☐	Kỹ năng	Gợi ý
☑	Khởi động MARS và tạo tệp mới	Nhấp đúp `Mars4_5.jar` (hoặc chạy `java -jar Mars4_5.jar`), sau đó chọn File → New.
☑	Kiểm tra các thiết lập dùng trong bài này	Trong menu Settings: bật Permit extended (pseudo) instructions and formats; bật Initialize Program Counter to global 'main' if defined; tắt Delayed branching; tắt Popup dialog for input syscalls.
☑	Assemble chương trình	Chọn Run → Assemble (hoặc biểu tượng trên thanh công cụ). MARS chuyển từ tab Edit sang tab Execute.
☑	Chạy toàn bộ chương trình	Chọn Run → Go. Văn bản do chương trình in ra xuất hiện trong khung Run I/O ở phía dưới.
☑	Thực thi từng lệnh một	Run → Step chạy lệnh kế tiếp; Run → Backstep hoàn tác một lệnh; Run → Reset đưa chương trình về trạng thái đầu. Phím tắt được ghi bên cạnh mỗi lệnh.
☑	Nhận biết các khung của tab Execute	Registers (bên phải), Text Segment (phía trên bên trái), Data Segment (bên dưới), Messages và Run I/O (phía dưới cùng).
☑	Chuyển giá trị giữa thập phân và hexa	Settings → Values displayed in hexadecimal.
☑	Lưu bài làm	File → Save as… và đặt tên theo quy tắc ở bên dưới.

> Lưu ý
>
> MARS có hai chế độ xem một chương trình: tab Edit, nơi gõ mã, và tab Execute, nơi chạy chương trình. Sau mỗi thay đổi trong trình soạn thảo, phải assemble lại. Assemble cũng đặt toàn bộ thanh ghi và bộ nhớ về trạng thái ban đầu.

Bước 0 – Kiểm tra nhanh

Gõ chương trình ở Mã nguồn 0 vào một tệp mới (gõ đúng nguyên văn), lưu lại, assemble và chạy. Ý nghĩa của từng dòng sẽ được giải thích trong Thí nghiệm 3; ở bước này chỉ cần xác nhận phần mềm hoạt động bình thường.

Mã nguồn 0

```
# Ma nguon 0 - kiem tra nhanh
.data
msg:    .asciiz "Xin chao, MARS!\n"

.text
.globl main
main:
    li   $v0, 4
    la   $a0, msg
    syscall
    li   $v0, 10
    syscall
```

Văn bản xuất hiện trong khung Run I/O:

```
Xin chao, MARS!
```

> **Quy tắc đặt tên tệp**
>
> Mỗi chương trình nộp bài được lưu với tên `IT012_Lab01_<MSSV>_<Phần>.asm`, ví dụ `IT012_Lab01_22520001_TN3.asm`. Dùng `TN2` hoặc `TN3` cho phần tương ứng. Đầu mỗi tệp phải có một khối chú thích gồm họ tên, MSSV và mục đích của chương trình trong một dòng.

THÍ NGHIỆM 1. LÀM QUEN VỚI THANH GHI (30 phút – 2 điểm)

Tình huống

Bộ xử lý trong công tơ thông minh không tính toán trực tiếp trên các số nằm xa trong bộ nhớ. Trước hết nó đưa các số vào một tập nhỏ các ô nhớ rất nhanh gọi là thanh ghi rồi tính toán ở đó. Trước khi lập trình cho công tơ, cần biết các thanh ghi.

Kiến thức nền: thanh ghi và bộ đếm chương trình

Bộ xử lý thực thi các lệnh lần lượt từng lệnh một. MIPS có 32 thanh ghi, mỗi thanh ghi rộng 32 bit (một *từ*, word). Mỗi thanh ghi có một số hiệu từ 0 đến 31 và một tên bắt đầu bằng `$`. Bảng 3 liệt kê các thanh ghi được dùng trong bài này.

Bộ xử lý còn có một thanh ghi đặc biệt là bộ đếm chương trình (Program Counter, `pc`), chứa địa chỉ của lệnh sẽ được thực thi tiếp theo. MARS hiển thị nội dung thanh ghi dưới dạng số hexa như `0x0000002A`: mỗi chữ số hexa biểu diễn 4 bit nên 8 chữ số tương ứng 32 bit.

Bảng 3. Các thanh ghi dùng trong bài thực hành

Tên	Số hiệu	Công dụng theo quy ước
$zero	0	Hằng số 0. Luôn chứa giá trị 0.
$v0–$v1	2–3	Kết quả; `$v0` còn chứa số hiệu dịch vụ của lời gọi hệ thống (Thí nghiệm 3).
$a0–$a3	4–7	Đối số; `$a0` còn chứa giá trị truyền cho lời gọi hệ thống (Thí nghiệm 3).
$t0–$t9	8–15, 24–25	Giá trị tạm thời.
$s0–$s7	16–23	Biến có giá trị được giữ lại lâu hơn.
$sp	29	Con trỏ ngăn xếp (dùng trong bài thực hành sau).
$ra	31	Địa chỉ trả về (dùng trong bài thực hành sau).

Bước 1.1 – Tìm các thanh ghi (8 phút)

Assemble Mã nguồn 0 nhưng chưa chạy. Quan sát khung Registers và hoàn thành Bảng 4 với số hiệu thanh ghi và giá trị ban đầu (hexa).

Bảng 4. Nội dung ban đầu của các thanh ghi

Thanh ghi	Số hiệu	Giá trị ban đầu (hexa)
$zero	0	0x00000000

Thanh ghi	Số hiệu	Giá trị ban đầu (hexa)
$v0	2	0x00000000
$a0	4	0x00000000
$t0	8	0x00000000
$t9	25	0x00000000
$s0	16	0x00000000
$sp	29	0x7fffeffc
pc	–	0x00400000

Câu 1.1 (Hiểu) So sánh giá trị ban đầu của `pc` với cột *Address* của dòng đầu tiên trong Text Segment. Nhận xét điều quan sát được và cho biết điều này nói gì về vai trò của bộ đếm chương trình.

Giá trị ban đầu của pc bằng cột Address của dòng đầu tiên trong Text Segment. Điều này cho thấy vai trò của PC là chứa địa chỉ của lệnh sắp được thực thi.

Bước 1.2 – Dự đoán, quan sát, giải thích (15 phút)

Mở một tệp mới, gõ Mã nguồn 1, lưu và assemble. Trước khi chạy bất cứ lệnh nào, điền vào cột DỰ ĐOÁN của Bảng 5 giá trị thập phân mà thanh ghi đích sẽ chứa sau mỗi lệnh.

Mã nguồn 1

```
# Ma nguon 1 - nhung buoc dau voi thanh ghi
.text
.globl main
main:
    addi $t0, $zero, 12      # $t0 = $zero + 12
    addi $t1, $zero, 30      # $t1 = $zero + 30
    add  $t2, $t0, $t1       # $t2 = $t0 + $t1
    addi $zero, $zero, 5     # thu dua $zero + 5 vao $zero
    add  $t3, $zero, $t2     # $t3 = $zero + $t2
    li    $v0, 10               # ket thuc chuong trinh (xem Thi
nghiem 3)
    syscall
```

Bây giờ dùng Run → Step để thực thi lần lượt năm lệnh đầu tiên. Sau mỗi lần Step, MARS tô sáng thanh ghi vừa thay đổi. Ghi lại giá trị quan sát được của thanh ghi đích và giá trị của `pc` sau lệnh đó. Dùng Backstep ít nhất một lần để thấy một lệnh được hoàn tác.

Bảng 5. Dự đoán và quan sát (thanh ghi: số thập phân; pc: số hexa)

Lệnh	Thanh ghi đích	Giá trị DỰ ĐOÁN	Giá trị QUAN SÁT	PC QUAN SÁT sau lệnh
addi $t0, $zero, 12	$t0	0x0000000c	0x0000000c	0x00400004
addi $t1, $zero, 30	$t1	0x0000001e	0x0000001e	0x00400008
add $t2, $t0, $t1	$t2	0x0000002a	0x0000002a	0x0040000c
addi $zero, $zero, 5	$zero	0x00000005	0x00000000	0x00400010
add $t3, $zero, $t2	$t3	0x0000002a	0x0000002a	0x00400014

Câu 1.2 (Vận dụng) Chuyển hiển thị sang hexa. Lúc này `$t2` hiển thị `0x0000002a`. Đổi số này sang thập phân bằng tay và trình bày phép tính.

Đổi sang thập phân ta được: 42.

Phép tính: 2 x 16 ^ 1 + 10 x 16 ^ 0

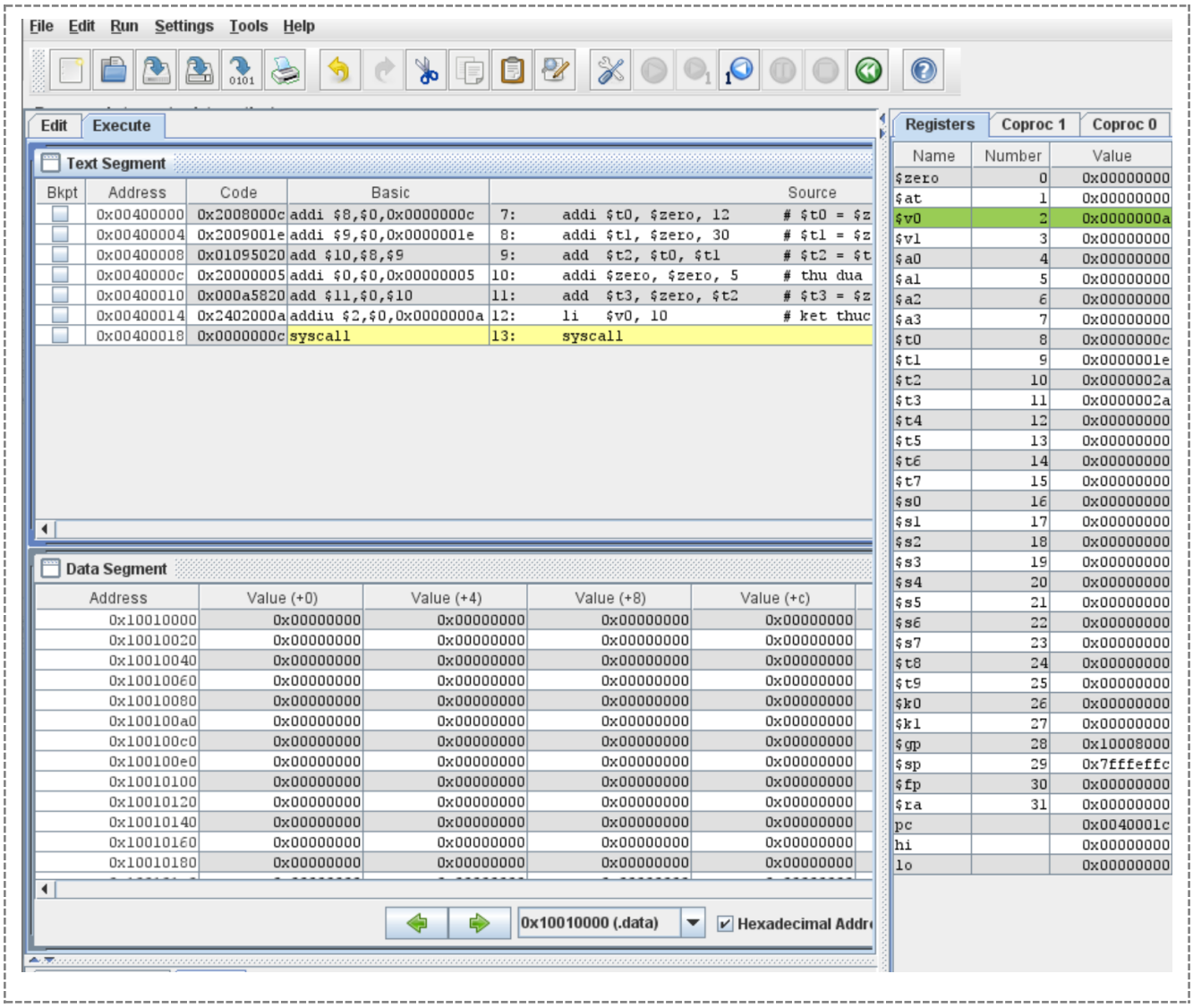
Câu 1.3 (Phân tích) Có một dự đoán không khớp với kết quả MARS hiển thị. Đó là lệnh nào? Hãy giải thích điều đã xảy ra và điều này cho biết gì về `$zero`.

Theo addi em tưởng zero + 5 = 5 nhưng thanh ghi $zero luôn bằng 0 và không thể bị ghi đè. $zero là hằng số, chỉ có thể đọc chứ không thể bị ghi đè lên.

Câu 1.4 (Phân tích) `pc` tăng thêm bao nhiêu sau mỗi lệnh? Mỗi lệnh MIPS chiếm một từ 32 bit trong bộ nhớ. Dùng thông tin này để giải thích con số quan sát được.

PC tăng 4 byte mỗi lệnh. Mỗi lệnh MIPS chiếm một từ 32 bit trong bộ nhớ mà mỗi byte bằng 8 bit nên 32 / 8 = 4 byte.

Ảnh chụp khung Registers sau lệnh thứ năm:


File Edit Run Settings Tools Help
Edit Execute
Text Segment
Bkpt Address Code Basic Source
0x00400000 0x2008000c addi $8,$0,0x0000000c 7: addi $t0, $zero, 12 # $t0 = $z
0x00400004 0x2009001e addi $9,$0,0x0000001e 8: addi $t1, $zero, 30 # $t1 = $z
0x00400008 0x01095020 add $10,$8,$9 9: add $t2, $t0, $t1 # $t2 = $t
0x0040000c 0x20000005 addi $0,$0,0x00000005 10: addi $zero, $zero, 5 # thu dua
0x00400010 0x000a5820 add $11,$0,$10 11: add $t3, $zero, $t2 # $t3 = $z
0x00400014 0x2402000a addiu $2,$0,0x0000000a 12: li $v0, 10 # ket thuc
0x00400018 0x0000000c syscall 13: syscall
Data Segment
Address Value (+0) Value (+4) Value (+8) Value (+c)
0x10010000 0x00000000 0x00000000 0x00000000 0x00000000
0x10010020 0x00000000 0x00000000 0x00000000 0x00000000
0x10010040 0x00000000 0x00000000 0x00000000 0x00000000
0x10010060 0x00000000 0x00000000 0x00000000 0x00000000
0x10010080 0x00000000 0x00000000 0x00000000 0x00000000
0x100100a0 0x00000000 0x00000000 0x00000000 0x00000000
0x100100c0 0x00000000 0x00000000 0x00000000 0x00000000
0x100100e0 0x00000000 0x00000000 0x00000000 0x00000000
0x10010100 0x00000000 0x00000000 0x00000000 0x00000000
0x10010120 0x00000000 0x00000000 0x00000000 0x00000000
0x10010140 0x00000000 0x00000000 0x00000000 0x00000000
0x10010160 0x00000000 0x00000000 0x00000000 0x00000000
0x10010180 0x00000000 0x00000000 0x00000000 0x00000000
0x10010000 (.data)
Hexadecimal Addr
Registers Coproc 1 Coproc 0
Name Number Value
$zero 0 0x00000000
$at 1 0x00000000
$v0 2 0x0000000a
$v1 3 0x00000000
$a0 4 0x00000000
$a1 5 0x00000000
$a2 6 0x00000000
$a3 7 0x00000000
$t0 8 0x0000000c
$t1 9 0x0000001e
$t2 10 0x0000002a
$t3 11 0x0000002a
$t4 12 0x00000000
$t5 13 0x00000000
$t6 14 0x00000000
$t7 15 0x00000000
$s0 16 0x00000000
$s1 17 0x00000000
$s2 18 0x00000000
$s3 19 0x00000000
$s4 20 0x00000000
$s5 21 0x00000000
$s6 22 0x00000000
$s7 23 0x00000000
$t8 24 0x00000000
$t9 25 0x00000000
$k0 26 0x00000000
$k1 27 0x00000000
$gp 28 0x10008000
$sp 29 0x7fffeffc
$fp 30 0x00000000
$ra 31 0x00000000
pc 0x0040001c
hi 0x00000000
lo 0x00000000

THÍ NGHIỆM 2. TỪ a = b + c ĐẾN $t0 = $t1 + $t2 (45 phút – 3 điểm)

Tình huống

Phần mềm nhúng (firmware) của công tơ phải tính điện năng tiêu thụ giữa hai lần ghi chỉ số. Trong ngôn ngữ cấp cao, lập trình viên viết `usage = reading2 - reading1`. Tuy nhiên bộ xử lý chỉ thực thi các lệnh gọi tên đúng ba thanh ghi. Thí nghiệm này cho thấy một câu lệnh được dịch từng bước như thế nào, cho đến tận con số mà máy thực sự thực thi.

Kiến thức nền: từ câu lệnh đến mã máy

Cùng một phép toán có thể được viết ở ba mức (Bảng 6). Chương trình dịch hợp ngữ sang mã máy gọi là assembler; MARS có sẵn một assembler.

Bảng 6. Ba mức biểu diễn của cùng một phép toán

Mức	Ví dụ	Người thực hiện
Câu lệnh ngôn ngữ cấp cao	a = b + c	lập trình viên
Lệnh hợp ngữ	add $t0, $t1, $t2	lập trình viên (hoặc trình biên dịch)
Mã máy	một số 32 bit, hiển thị bằng 8 chữ số hexa	assembler

Biến trở thành thanh ghi

Bộ xử lý không có biến được đặt tên. Ta tự quy ước thanh ghi nào đại diện cho biến nào, ví dụ `a` → `$t0`, `b` → `$t1`, `c` → `$t2`. Lệnh `add` có dạng

```
add  đích, nguồn1, nguồn2
```

nên `a = b + c` trở thành `add $t0, $t1, $t2`. Một lệnh thực hiện một phép toán trên ba thanh ghi; biểu thức dài hơn cần nhiều lệnh. Ví dụ, `x = p + q - r` với `x`→`$t0`, `p`→`$t1`, `q`→`$t2`, `r`→`$t3` cần hai lệnh:

```
add  $t0, $t1, $t2      # x = p + q
sub  $t0, $t0, $t3      # x = x - r
```

Để đưa một hằng số vào thanh ghi, cộng hằng số đó với `$zero`: `addi $t1, $zero, 7` đặt `$t1` bằng 7. Để sao chép một thanh ghi, cộng `$zero` vào nó: `add $t2, $t1, $zero` sao chép `$t1` vào `$t2`.

Bước 2.1 – Phép cộng đầu tiên (8 phút)

Gõ Mã nguồn 2, lưu và assemble. DỰ ĐOÁN giá trị cuối cùng của `$t0` trước khi chạy.

Mã nguồn 2

```
# Ma nguon 2 - a = b + c
.text
.globl main
main:
    addi $t1, $zero, 7        # b = 7
    addi $t2, $zero, 5        # c = 5
    add  $t0, $t1, $t2        # a = b + c
    li   $v0, 10
    syscall
```

DỰ ĐOÁN – giá trị cuối cùng của `$t0` (thập phân):

0x0000000c

QUAN SÁT – thực thi ba lệnh bằng Step và ghi lại ba thanh ghi sau mỗi lệnh.

Bảng 7. Các thanh ghi sau mỗi lệnh của Mã nguồn 2

Sau khi thực thi	$t0	$t1	$t2
addi $t1, $zero, 7	0x00000000	0x00000007	0x00000000
addi $t2, $zero, 5	0x00000000	0x00000007	0x00000005
add $t0, $t1, $t2	0x0000000c	0x00000007	0x00000005

Bước 2.2 – Xem mã máy (12 phút)

Chọn tab Execute và quan sát Text Segment. Mỗi dòng có các cột Address, Code, Basic và Source: *Source* là văn bản đã gõ, *Basic* là lệnh thật mà bộ xử lý thực thi, còn *Code* là từ mã máy 32 bit ở dạng hexa. Tìm dòng của `add $t0, $t1, $t2` và ghi lại địa chỉ và mã của nó.

> **Kiến thức nền: định dạng lệnh R-type**
>
> Các lệnh như `add` và `sub` dùng định dạng R, trong đó 32 bit được chia thành sáu trường:
>
> | op | rs | rt | rd | shamt | funct |
> |---|---|---|---|---|---|
> | 6 bit | 5 bit | 5 bit | 5 bit | 5 bit | 6 bit |

> `op` là mã lệnh; `rs` và `rt` là số hiệu của thanh ghi nguồn thứ nhất và thứ hai; `rd` là số hiệu của thanh ghi đích; `shamt` là lượng dịch (bằng 0 với `add`); `funct` chọn phép toán cụ thể. Để đọc một mã hexa, thay mỗi chữ số bằng 4 bit: 0=0000, 1=0001, 2=0010, 3=0011, 4=0100, 5=0101, 6=0110, 7=0111, 8=1000, 9=1001, A=1010, B=1011, C=1100, D=1101, E=1110, F=1111.

Địa chỉ của dòng:

0x00400008

Mã của dòng (hexa):

0x012a4020

Bảng 8. Các trường của lệnh `add $t0, $t1, $t2`

	op	rs	rt	rd	shamt	funct
Nhị phân	00000	01001	01010	01000	00000	10000
Thập phân	0	9	10	8	0	32

Câu 2.1 (Phân tích) So sánh giá trị thập phân của `rs`, `rt` và `rd` với số hiệu thanh ghi trong Bảng 3. Mỗi trường chứa toán hạng nào của câu lệnh hợp ngữ? Ba thanh ghi được viết theo thứ tự nào trong câu lệnh hợp ngữ, và các trường tương ứng xuất hiện theo thứ tự nào trong từ mã máy?

Giá trị thập phân của rs, rt và rd bằng với số hiệu thanh ghi trong Bảng 3.

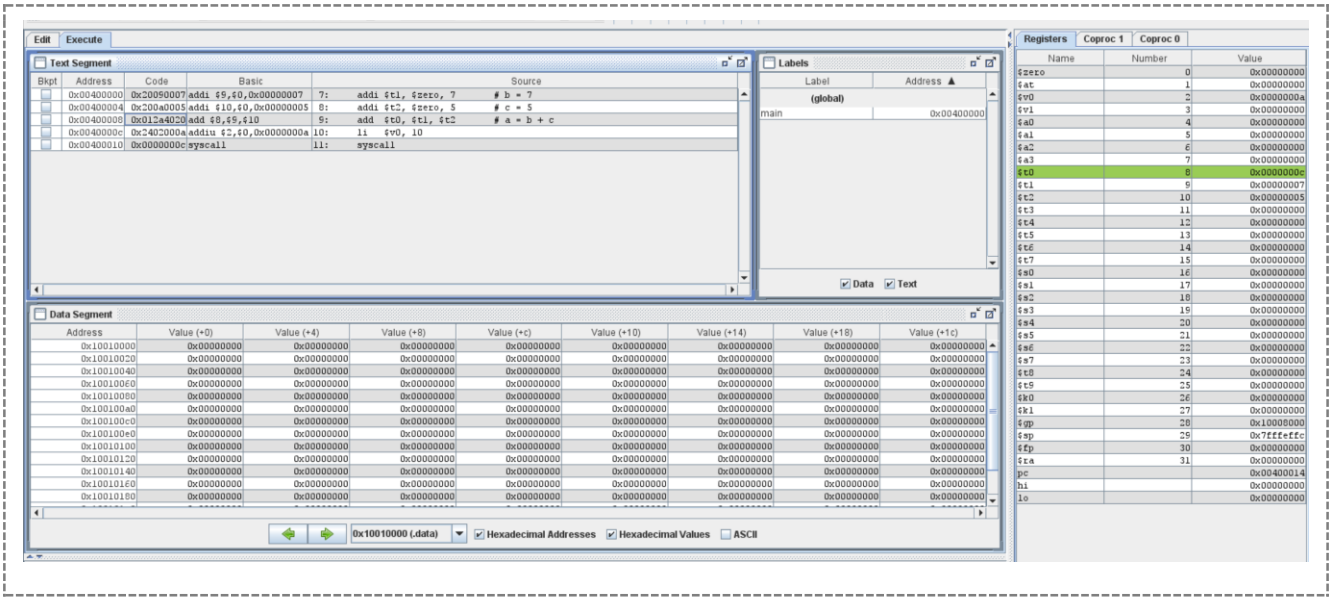
rs chứa thanh ghi nguồn thứ nhất, rt chứa thanh ghi nguồn thứ hai, rd chứa thanh ghi đích.

Trong câu lệnh hợp ngữ add $t0, $t1, $t2, ba thanh ghi được viết theo thứ tự đích, nguồn 1, nguồn 2

Câu 2.2 (Hiểu) Xem dòng của `li $v0, 10`. So sánh cột *Source* và cột *Basic* của dòng này. `li` có phải là lệnh mà phần cứng tự thực thi không? Assembler đã làm gì với nó?

Cột Source hiển thị addiu $2, $0, 0x0000000a còn Basic hiển thị li $v0, 10. Đây là lệnh mà phần cứng tự thực thi để Assembler thoát chương tr

Ảnh chụp Text Segment hiển thị dòng của `add $t0, $t1, $t2`:



Bước 2.3 – Điện năng tiêu thụ của một hộ gia đình (12 phút)

Mã nguồn 3 nạp bốn chỉ số công tơ (đơn vị kWh) ghi vào đầu tháng 1, đầu tháng 2, đầu tháng 3 và cuối tháng 3. Hoàn thành chương trình sao cho `$t0`, `$t1` và `$t2` chứa điện năng tiêu thụ của tháng 1, 2 và 3, còn `$t3` chứa tổng. Chỉ dùng `add` và `sub`.

Mã nguồn 3

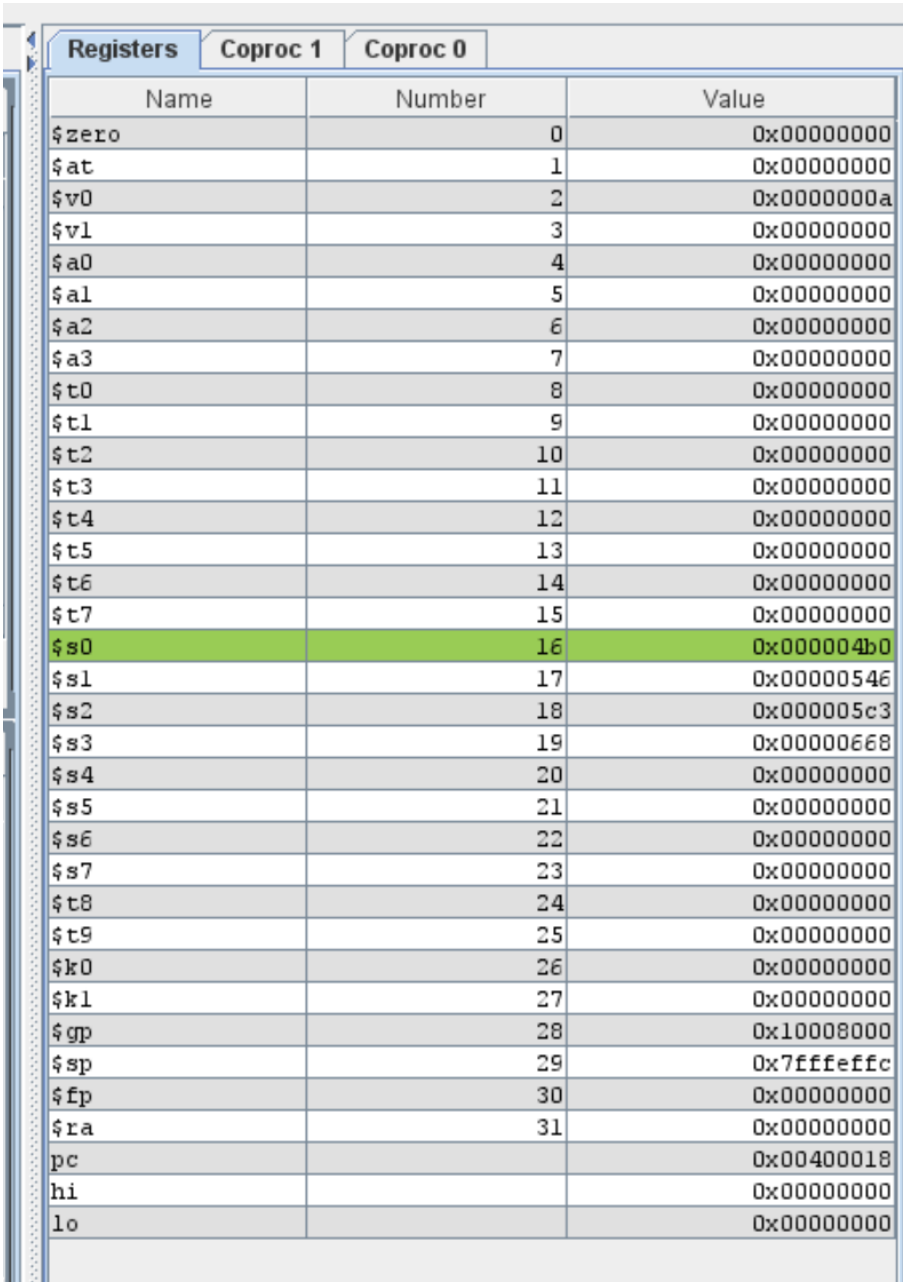
```
# Ma nguon 3 - dien nang tieu thu cua mot ho gia dinh (so lieu gia
lap, kWh)
# Bien       Thanh ghi
# reading0   $s0   dau thang 1
# reading1   $s1   dau thang 2
# reading2   $s2   dau thang 3
# reading3   $s3   cuoi thang 3
# usage1     $t0   dien nang thang 1
# usage2     $t1   dien nang thang 2
# usage3     $t2   dien nang thang 3
# total      $t3   usage1 + usage2 + usage3
.text
.globl main
main:
    addi $s0, $zero, 1200
    addi $s1, $zero, 1350
    addi $s2, $zero, 1475
    addi $s3, $zero, 1640

    # >>> usage1 = reading1 - reading0
    # >>> usage2 = reading2 - reading1
    # >>> usage3 = reading3 - reading2
    # >>> total = usage1 + usage2 + usage3   (hai lenh add)

    li   $v0, 10
    syscall
```

> **Kiểm tra kết quả**
>
> Với các chỉ số này, sau khi chương trình chạy xong, `$t0`, `$t1`, `$t2` và `$t3` phải chứa lần lượt 150, 125, 165 và 440.

Ảnh chụp khung Registers sau khi chương trình chạy xong:



Registers | Coproc 1 | Coproc 0

Name	Number	Value
$zero	0	0x00000000
$at	1	0x00000000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x00000000
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000000
$t1	9	0x00000000
$t2	10	0x00000000
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x000004b0
$s1	17	0x00000546
$s2	18	0x000005c3
$s3	19	0x00000668
$s4	20	0x00000000
$s5	21	0x00000000
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x00000000
$t9	25	0x00000000
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00000000
pc		0x00400018
hi		0x00000000
lo		0x00000000

Câu 2.3 (Phân tích) Tính `reading3 - reading0` bằng tay và so sánh với `total`. Vì sao hai số này phải bằng nhau? Giải thích dựa trên ý nghĩa của các chỉ số công tơ.

Hai số phải bằng nhau vì mỗi chỉ số công tơ là số tích lũy (chỉ tăng theo thời gian), nên mức tiêu thụ trong các tháng phải bằng hiệu những lần đọc liên tiếp.

Bước 2.4 – Chuyển câu lệnh thành lệnh hợp ngữ (13 phút)

Bảng 9 liệt kê bốn câu lệnh. Trong tất cả các câu lệnh, các biến `y`, `z` và `w` nằm trong `$s0`, `$s1` và `$s2` và có giá trị ban đầu lần lượt là 8, 5 và 3. Viết các lệnh MIPS cho từng câu lệnh vào ô nền vàng, dùng thanh ghi đích đã cho. Khi cần kết quả trung gian, dùng `$t5` làm thanh ghi tạm. DỰ ĐOÁN kết quả của từng câu lệnh bằng tay trước khi kiểm tra.

Bảng 9. Các câu lệnh cần chuyển đổi

Câu lệnh	Thanh ghi đích	Các lệnh MIPS của sinh viên	DỰ ĐOÁN (thập phân)	QUAN SÁT (thập phân)
x1 = y + z + w	$t0	Add $t5, $s0, $s1 Add $t0, $t5, $s2	16	16
x2 = (y + z) - (w + 5)	$t1	Add $t5, $s0, $s1 Addi $t1, $s2, 5 Sub $t1, $t5, $t1	5	5
`x3 = y` (sao chép)	$t2	Add $t2, $zero, $s0	8	8
x4 = z - y	$t3	Sub $t3, $s1, $s0	-3	-3

Để kiểm tra đáp án, tạo một tệp mới với các dòng bên dưới, thêm các lệnh của sinh viên tại vị trí đánh dấu, chạy chương trình và đọc `$t0–$t3`.

Mã nguồn 4

```
# Ma nguon 4 - chuyen cau lenh thanh lenh hop ngu
.text
.globl main
main:
    addi $s0, $zero, 8        # y = 8
    addi $s1, $zero, 5        # z = 5
    addi $s2, $zero, 3        # w = 3

    # >>> cac lenh cho x1, x2, x3 va x4

    li   $v0, 10
    syscall
```

Câu 2.4 (Phân tích) Giá trị của `x4` là số âm. Hãy viết `$t3` ở dạng hexa và dạng thập phân. Các chữ số hexa đầu tiên của một số âm có đặc điểm gì, và điều đó gợi ý gì về cách bộ xử lý lưu số âm?

Dạng thập phân: -3

Dạng hexa: 0xfffffffd

Các chữ số hexa đầu tiên của số âm là 0xf, điều này cho thấy bộ xử lý lưu số âm theo biểu diễn bù hai bit ngoài cùng bên trái là bit dấu.

THÍ NGHIỆM 3. NHẬP, XUẤT VÀ LỜI GỌI HỆ THỐNG (50 phút – 3,5 điểm)

> **Tình huống**
>
> Kỹ thuật viên nhập các chỉ số vào công tơ thông minh, và công tơ hiển thị điện năng tiêu thụ cho khách hàng. Một chương trình chạy trên bộ xử lý không tự có màn hình hay bàn phím: nó phải nhờ hệ thống đọc và in thay cho mình. Yêu cầu này được gọi là lời gọi hệ thống (system call).

> **Kiến thức nền: bộ nhớ chương trình và văn bản**
>
> Một chương trình MIPS có hai phần. Phần `.text` chứa các lệnh. Phần `.data` chứa dữ liệu mà chương trình sử dụng, chẳng hạn các thông báo. Trong MARS, phần dữ liệu bắt đầu tại địa chỉ `0x10010000`.
>
> Nhãn (label) là tên của một địa chỉ. Chỉ thị `.asciiz "văn bản"` lưu các ký tự của thông báo, theo sau là thêm một byte có giá trị 0 để đánh dấu kết thúc thông báo. Lệnh giả `la $a0, nhãn` đặt địa chỉ của nhãn vào `$a0`.

> **Kiến thức nền: cách thực hiện một lời gọi hệ thống**
>
> 1. Đặt số hiệu dịch vụ vào `$v0`. 2. Đặt đối số (nếu có) vào `$a0`. 3. Thực hiện `syscall`. 4. Nếu dịch vụ trả về kết quả, đọc kết quả từ `$v0`.
>
> Trong MARS, trình mô phỏng đóng vai trò hệ điều hành và thực hiện dịch vụ. Bảng 10 liệt kê bốn dịch vụ dùng trong bài thực hành này.

Bảng 10. Các dịch vụ lời gọi hệ thống dùng trong bài thực hành

Dịch vụ	$v0	Đối số	Kết quả
In số nguyên	1	`$a0` = số nguyên cần in	–
In chuỗi	4	`$a0` = địa chỉ của chuỗi kết thúc bằng 0	–
Đọc số nguyên	5	–	số nguyên người dùng gõ, trả về trong `$v0`
Kết thúc chương trình	10	–	–

Bước 3.1 – In văn bản và một số (12 phút)

Gõ Mã nguồn 5, lưu và assemble. DỰ ĐOÁN chính xác nội dung sẽ xuất hiện trong khung Run I/O.

Mã nguồn 5

```
# Ma nguon 5 - in mot thong bao va mot so nguyen
.data
msg:    .asciiz "Dien nang (kWh): "
nl:     .asciiz "\n"

.text
.globl main
main:
    addi $t0, $zero, 150
    li   $v0, 4              # dich vu 4: in chuoi
    la   $a0, msg            # doi so: dia chi cua msg
    syscall
    li   $v0, 1              # dich vu 1: in so nguyen
    add  $a0, $t0, $zero     # doi so: so can in
    syscall
    li   $v0, 4              # in xuong dong
    la   $a0, nl
    syscall
    li   $v0, 10             # dich vu 10: ket thuc
    syscall
```

DỰ ĐOÁN – nội dung trong khung Run I/O:

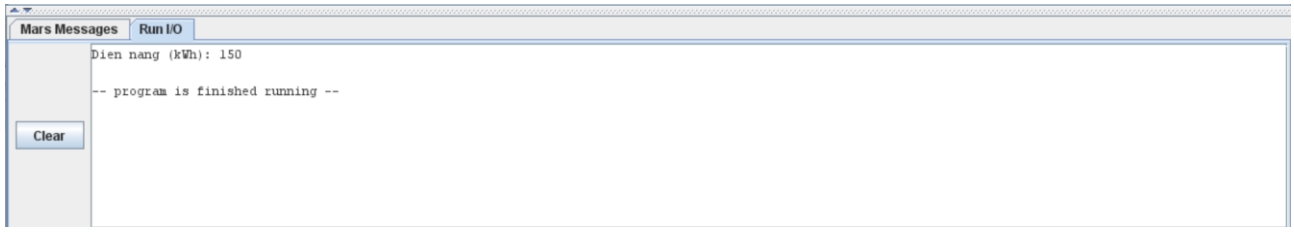
```
Dien nang (kWh) : 150
```

QUAN SÁT – chạy chương trình. Sau đó mở Settings → Show Labels Window (symbol table) và đọc địa chỉ của hai nhãn.

Bảng 11. Địa chỉ của các nhãn

Nhãn	Địa chỉ (hexa)
msg	0x10010000
nl	0x10010012

Ảnh chụp khung Run I/O sau khi chương trình chạy xong:



Câu 3.1 (Hiểu) Thanh ghi nào cho hệ thống biết *cần dịch vụ nào*, và thanh ghi nào mang *nội dung cần in*? Nếu đổi số hiệu dịch vụ thì ý nghĩa của `$a0` thay đổi thế nào?

> Thanh ghi $v0 cho hệ thống biết cần dịch vụ nào. Thanh ghi $a0 mang nội dung cần in. Nếu thay đổi số hiệu dịch vụ thì $a0. Ý nghĩa của $a0 phụ thuộc vào số hiệu dịch vụ trong $v0: với dịch vụ 4, $a0 là địa chỉ của chuỗi cần in; với dịch vụ 1, $a0 là giá trị số nguyên cần in. Cùng một thanh ghi nhưng được hiểu là địa chỉ hay là số tùy theo $v0.

Câu 3.2 (Vận dụng) Lấy địa chỉ của `nl` trừ đi địa chỉ của `msg`. Đếm số ký tự của thông báo `Dien nang (kWh):` (kể cả dấu cách) rồi so sánh với kết quả vừa tính. Giải thích sự chênh lệch giữa hai số.

> n1- msg = 18. Chênh lệch 1 là do chỉ thị .asciiz tự động thêm một byte ký tự kết thúc chuỗi (null, giá trị 0) vào cuối chuỗi. Vì vậy chuỗi chiếm 17 byte ký tự + 1 byte null = 18 byte, và nl bắt đầu ngay sau byte null đó. Byte null này cũng là dấu hiệu để syscall 4 biết khi nào dừng in.

Bước 3.2 – Đọc hai số và tính điện năng tiêu thụ (15 phút)

Mã nguồn 6 đọc chỉ số công tơ kỳ trước từ bàn phím và lưu vào `$s0`. Hoàn thành chương trình để nó đọc thêm chỉ số hiện tại vào `$s1`, tính `usage = current - previous` vào `$t0`, và in kết quả theo dạng nêu bên dưới mã nguồn.

Mã nguồn 6

```
# Ma nguon 6 - dien nang tieu thu giua hai chi so
.data
prompt1: .asciiz "Nhap chi so cu (kWh): "
prompt2: .asciiz "Nhap chi so moi (kWh): "
label:   .asciiz "Dien nang (kWh): "
newline: .asciiz "\n"

.text
.globl main
main:
    # --- doc chi so cu vao $s0 ---
    li   $v0, 4
    la   $a0, prompt1
    syscall
    li   $v0, 5              # dich vu 5: doc so nguyen, ket qua
o $v0
    syscall
    add  $s0, $v0, $zero     # giu gia tri trong $s0

    # >>> doc chi so moi vao $s1 (in prompt2, roi dich vu 5)

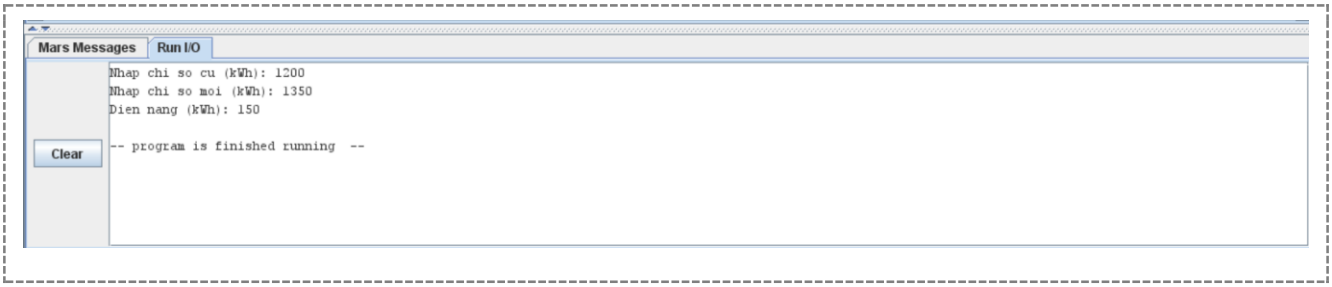
    # >>> usage = current - previous, vao $t0

    # >>> in label, roi usage (dich vu 1), roi newline

    li   $v0, 10
    syscall
```

> **Kiểm tra kết quả**
>
> Chạy chương trình và gõ `1200` rồi `1350` khi được yêu cầu. Dòng cuối cùng trong khung Run I/O phải là `Dien nang (kWh): 150`. Gõ từng số vào khung Run I/O và nhấn Enter.

Ảnh chụp khung Run I/O của lần kiểm tra này:



Bước 3.3 – Tổng hợp ba tháng với dữ liệu của riêng sinh viên (15 phút)

Mở rộng chương trình để đọc bốn chỉ số theo thứ tự (reading0 đến reading3), sau đó in điện năng tiêu thụ của từng tháng và tổng. Bắt đầu từ phần `.data` bên dưới và giữ nguyên kế hoạch thanh ghi của Mã nguồn 3 (`$s0–$s3` cho các chỉ số, `$t0–$t3` cho các kết quả). Kết quả in ra phải có dạng:

```
Thang 1: <usage1>
Thang 2: <usage2>
Thang 3: <usage3>
Tong:    <total>
```

Mã nguồn 7 (phần dữ liệu)

```
.data
prompt: .asciiz "Nhap chi so (kWh): "
m1:     .asciiz "Thang 1: "
m2:     .asciiz "Thang 2: "
m3:     .asciiz "Thang 3: "
tot:    .asciiz "Tong:    "
nl:     .asciiz "\n"
```

Trước hết kiểm tra chương trình với các chỉ số 1200, 1350, 1475 và 1640; bốn kết quả phải là 150, 125, 165 và 440. Sau đó lấy D là chữ số cuối cùng của MSSV và tra chỉ số riêng trong Bảng 12. DỰ ĐOÁN bốn kết quả bằng tay, chạy chương trình với chỉ số của mình và ghi lại điều quan sát được.

Bảng 12. Chỉ số riêng (D = chữ số cuối cùng của MSSV)

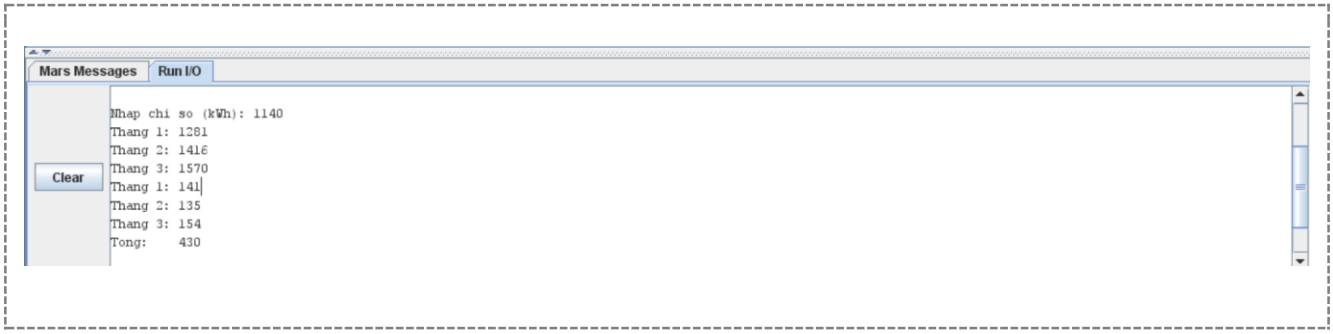
D	reading0	reading1	reading2	reading3
0	1000	1120	1220	1360
1	1020	1143	1248	1390
2	1040	1166	1276	1420

D	reading0	reading1	reading2	reading3
3	1060	1189	1304	1450
4	1080	1212	1332	1480
5	1100	1235	1360	1510
6	1120	1258	1388	1540
7	1140	1281	1416	1570
8	1160	1304	1444	1600
9	1180	1327	1472	1630

Bảng 13. Kết quả với chỉ số riêng

D của tôi	Kết quả	Tháng 1	Tháng 2	Tháng 3	Tổng
7	DỰ ĐOÁN	141	135	154	430
7	QUAN SÁT	141	135	154	430

Ảnh chụp khung Run I/O với chỉ số riêng của sinh viên:



Bước 3.4 – Cố ý làm sai (8 phút)

Lập trình viên học được rất nhiều từ lỗi. Với mỗi thí nghiệm trong Bảng 14, DỰ ĐOÁN điều sẽ xảy ra, thực hiện thay đổi, chạy chương trình và QUAN SÁT kết quả. Sau đó khôi phục lại chương trình.

Bảng 14. Các thí nghiệm có kiểm soát

Thí nghiệm	DỰ ĐOÁN	QUAN SÁT
(a) Chạy chương trình ở Bước 3.2 và nhập các chỉ số sai thứ tự: `1350` trước, rồi `1200`.	Chương trình sẽ cho ra số âm ở tháng 1	Chương trình cho số âm ở tháng 1: -150, những lệnh còn lại vẫn chạy như bình thường
(b) Trong Mã nguồn 5, xóa hai lệnh cuối cùng (`li $v0, 10` và `syscall`) rồi chạy.	Chương trình vẫn chạy nhưng sẽ không kết thúc	chương trình vẫn chạy bình thường, nhưng sau khi in xong, chương trình không chủ động yêu cầu hệ điều hành kết thúc bằng syscall 10.
(c) Trong Mã nguồn 5, đổi số hiệu dịch vụ trước lệnh in số nguyên từ `1` thành `4`, rồi chạy.	Chương trình bị lỗi và không chạy được	Chương trình in địa chỉ chuỗi ký tự và báo lỗi không chạy được

Câu 3.3 (Phân tích) Giải thích quan sát của thí nghiệm (c). Hệ thống đã cố coi giá trị nào là địa chỉ, và vì sao chương trình bị lỗi?

Chương trình không còn in giá trị số nguyên trong thanh ghi $t0 mà coi giá trị trong $a0 là địa chỉ của chuỗi ký tự để in ra nên bị lỗi nhập giá trị.

PHẦN 4. TỔNG KẾT VÀ NỘP BÀI (15 phút – 1,5 điểm)

Câu 4.1 (Hiểu) Bằng lời của mình, mô tả câu lệnh `a = b + c` trở thành thứ mà bộ xử lý có thể thực thi như thế nào. Sử dụng các thuật ngữ *thanh ghi*, *lệnh hợp ngữ*, *assembler* và *mã máy*.

Câu lệnh a = b + c được chuyển thành lệnh hợp ngữ bằng cách sử dụng các lệnh để nạp giá trị của b, c vào thanh ghi, thực hiện phép cộng rồi lưu kết quả vào a. Sau đó, assembler chuyển lệnh hợp ngữ thành mã máy gồm các bit 0 và 1 để bộ xử lý giải mã và thực thi.

Addi $s0, $zero, a

Addi $s1, $zero, b

Addi $s2, $zero, c

Add $s0, $s1, $s2

Câu 4.2 (Vận dụng) Viết các lệnh in số nguyên đang nằm trong `$t5` rồi in xuống dòng, giả sử trong phần `.data` đã có một chuỗi gắn nhãn `nl`.

```
li  $v0, 1
add $a0, $t5, $zero
syscall
li  $v0, 4
la  $a0, nl
syscall
```

Câu 4.3 (Đánh giá) Chương trình ở Bước 3.3 chứa nhiều khối lệnh gần giống nhau. Trong hai hoặc ba câu, hãy đề xuất một tính năng giúp chương trình như vậy ngắn gọn hơn và cho biết bộ xử lý cần có khả năng gì để hỗ trợ tính năng đó.

Có thể sử dụng hàm (procedure) để gom các khối lệnh giống nhau thành một đoạn mã và gọi lại khi cần, giúp chương trình ngắn gọn, dễ đọc và dễ bảo trì hơn. Để hỗ trợ tính năng này, bộ xử lý cần có lệnh gọi hàm, lệnh quay về và cơ chế lưu địa chỉ quay về, chẳng hạn thanh ghi $ra trong MIPS.

Bảng 15. Danh sách kiểm tra khi nộp bài

☐	**Nội dung**
☑	Các tệp IT012_Lab01_<MSSV>_TN2.asm và IT012_Lab01_<MSSV>_TN3.asm đã được lưu đúng tên.

☐	**Nội dung**
☑	Mỗi tệp bắt đầu bằng khối chú thích (họ tên, MSSV, mục đích) và mọi chương trình đều assemble không lỗi.
☑	Mọi dấu `# >>>` đã được thay bằng mã của sinh viên và kế hoạch dùng thanh ghi được ghi trong chú thích.
☑	Tài liệu này đã hoàn chỉnh: mọi ô nền vàng đã được điền và mọi khung nét đứt đã có ảnh chụp màn hình.
☑	Các tệp `.asm` đã được đẩy lên kho mã GitHub của sinh viên, và liên kết đến kho mã đã được ghi vào bảng thông tin ở trang đầu.
☑	Tài liệu và liên kết kho mã được nộp theo hình thức giảng viên thông báo.

PHỤ LỤC. TỔNG HỢP CÁC LỆNH

Bảng 16. Các lệnh dùng trong bài thực hành

Lệnh	Ví dụ	Ý nghĩa
add	add $t0, $t1, $t2	$t0 = $t1 + $t2
sub	sub $t0, $t1, $t2	$t0 = $t1 - $t2
addi	addi $t0, $t1, 20	$t0 = $t1 + 20
syscall	syscall	yêu cầu hệ thống thực hiện dịch vụ có số hiệu nằm trong `$v0`
`li` (lệnh giả)	li $t0, 7	`$t0 = 7`; assembler biến nó thành một lệnh thật
`la` (lệnh giả)	la $a0, label	`$a0` = địa chỉ của `label`; assembler biến nó thành các lệnh thật